🚀 GreenDevCorp - Infraestructura como Código (IaC)

Bienvenido al repositorio de infraestructura de GreenDevCorp. Este proyecto contiene la automatización completa para desplegar un servidor de desarrollo seguro, escalable y monitorizado basado en Debian 12, utilizando únicamente scripts de Bash y herramientas nativas de Linux.

🗂 Estructura del Repositorio

- `/week_1`: Configuración base (Usuarios, SSH, Firewall UFW).
- `/week_2`: Despliegue de servicios (Nginx) y auto-recuperación (Systemd).
- `/week_3`: Gestión de recursos (Cgroups) y monitorización de procesos.
- `/week_4`: Entorno colaborativo seguro (SGID, Sticky Bits, PAM limits).
- `/week_5`: Almacenamiento (ext4, fstab) y copias de seguridad incrementales (Rsync).
- `/docs`: Documentación técnica, Runbooks y Diseño de Arquitectura.
- `bootstrap.sh`: Script semilla para el despliegue automatizado "One-Click".

🛠 Cómo desplegar desde cero (Quickstart)

Si tienes un servidor Debian 12 completamente limpio, puedes levantar toda la infraestructura ejecutando este único comando como usuario `root`:

```bash curl -sL https://raw.githubusercontent.com/TU_USUARIO/scripting_gsx/main/bootstrap.sh | bash ```
(Nota: Cambia TU_USUARIO por tu usuario real de GitHub).

📖 Documentación del Sistema

Toda la documentación operativa y de diseño se encuentra en la carpeta `/docs`:

- 📘 Manual de Operaciones (Runbook)
- 🏗 Diseño de Arquitectura y Escalabilidad
- 🚑 Recuperación de Desastres
- 🧠 Ensayo de Reflexión

🏗 Documentación de Arquitectura y Diseño

Este documento detalla las decisiones de diseño, trade-offs y la planificación futura para la infraestructura del servidor de GreenDevCorp.

Decisiones de Diseño por Fases

Week 1: Foundational Server Administration

Para la configuración base, se optó por un enfoque modular e idempotente.

- **Estructura de Directorios (FHS):** Se utilizó `/opt/admin/` para almacenar scripts de administración en lugar de los directorios `/home`. Esto cumple con el estándar de jerarquía (FHS) para software que afecta a todo el sistema.
```

- **Seguridad SSH:** Se desactiva el acceso root por defecto, obligando al uso de usuarios sin privilegios que deben escalar permisos mediante `sudo`, garantizando la trazabilidad en los logs.

Week 2: Services, Observability & Automation

El objetivo es la alta disponibilidad y la prevención de fallos silenciosos.

- **Auto-recuperación (Drop-in):** Para hacer que Nginx sea resiliente, se creó un *drop-in* en `/etc/systemd/system/nginx.service.d/`. Se descartó modificar directamente `/lib/systemd/system/nginx.service` porque las actualizaciones de `apt` sobrescribirían nuestros cambios.
- **Systemd Timers vs Cron:** Se decidió abandonar `cron` a favor de `systemd timers`. La razón es la observabilidad: los timers se integran nativamente con `journald`, permitiendo registrar la salida estándar (stdout/stderr) directamente en los logs del sistema.

Week 3: Process Management & Resource Control

La visibilidad en los procesos es fundamental para diagnosticar cuellos de botella.

- **Escalabilidad Segura (cgroups):** Para evitar que procesos desbocados monopolicen el sistema, empaquetamos las cargas de trabajo en servicios systemd aplicando `CPUQuota=20%` y `MemoryMax=50M`.
- **Graceful Shutdown:** Se diseñaron scripts que responden a señales POSIX (`SIGTERM`, `SIGINT`), permitiendo limpiar procesos hijos antes de salir, en lugar de usar `SIGKILL` (fuerza bruta).

Week 4: Collaborative Environment & Access Control

El reto era permitir la colaboración sin comprometer la seguridad.

- **Trade-off: Standard Permissions vs ACLs:** Se decidió usar permisos estándar de Linux (SGID y Sticky Bits) en el directorio `/shared` en lugar de Listas de Control de Acceso (ACLs). Las ACLs son más granulares, pero los permisos estándar son más universales, fáciles de auditar y menos propensos a errores de configuración.
- **Protección de Datos:** El **Sticky Bit** (1770) garantiza que un desarrollador solo pueda borrar los archivos que él mismo ha creado, protegiendo el trabajo del resto del equipo.
- **Límites PAM:** Se implementaron límites en `/etc/security/limits.d/` para restringir el número máximo de procesos por usuario, previniendo bombas lógicas (fork bombs) accidentales.

Week 5: Storage & Backup Strategy

- **Trade-off: Incrementales vs Completos:** Hemos abandonado los backups completos (`tar`) a favor de incrementales con Hard Links usando `rsync`. Esto ahorra muchísimo espacio en disco (0 bytes extra para archivos no modificados), aunque requiere más CPU para calcular metadatos.
- **El Principio 3-2-1:** Los datos viven en el disco `sda` (1), el backup local en `sdb` (2), preparándolo para un offsite mediante NFS (3).

🚀 Future Planning: Escalabilidad del Sistema

¿Cómo escalaría el sistema a 20 desarrolladores?

El sistema actual soportaría 20 usuarios con pequeños ajustes:

1. **Gestión de Usuarios:** La creación manual de usuarios empezaría a ser ineficiente. Se implementaría **Ansible** para automatizar el aprovisionamiento de cuentas de forma masiva.
2. **Almacenamiento:** El disco virtual de 5GB se llenaría rápido. Sería necesario migrar `/home/greendevcorp/shared` a un volumen LVM para redimensionar el disco en caliente sin apagar el servidor.
3. **Recursos:** Habría que revisar los límites PAM y Cgroups para asegurar que los 20 usuarios concurrentes no agotan la RAM física del servidor.

¿Cómo escalaría el sistema a 100 desarrolladores?

Para 100 personas, la arquitectura de "un solo servidor monolítico" es insostenible y un punto único de fallo (SPOF).

1. **Identidad Centralizada:** Migraríamos los usuarios locales a un servidor de identidades como **LDAP/FreeIPA** o Active Directory.
2. **Almacenamiento Desacoplado:** El directorio `/shared` pasaría a estar alojado en una red NAS (NFS) dedicada, separando la computación del almacenamiento.
3. **Entornos Efímeros:** En lugar de entrar por SSH a un servidor compartido, la infraestructura migraría a contenedores (**Docker / Kubernetes**), donde cada desarrollador levanta su entorno aislado que se destruye al terminar.

🚑 Disaster Recovery & Verified Backup (DRP)

Este documento detalla el procedimiento de recuperación de datos para el servidor de GreenDevCorp, así como la evidencia de las pruebas de restauración.

📊 Tiempos Objetivo de Recuperación

- **RPO (Recovery Point Objective):** 24 horas. Los backups automatizados vía systemd timers se ejecutan cada madrugada a las 02:00 AM. En el peor escenario de pérdida total del disco principal, la pérdida máxima de datos es de un día de trabajo.
- **RTO (Recovery Time Objective):** < 5 minutos. Al utilizar `rsync` con copias exactas y hard links en un disco virtual secundario (`/mnt/data_vault`), la recuperación no requiere descompresión de archivos pesados. Es una copia directa bloque a bloque.

🛠️ Procedimiento de Recuperación Completo (Full Recovery)

En caso de pérdida de datos en el entorno de desarrollo compartido (`/home/greendevcorp/shared`), sigue estos pasos:

1. Identificar el último backup válido

Los backups se almacenan en el disco secundario `/dev/sdb`, montado en `/mnt/data_vault`. ```bash

Listar las copias disponibles ordenadas por fecha

sudo ls -l /mnt/data_vault/backups/ ``` *(Nota: Siempre existirá una carpeta llamada `latest` que apunta mediante un enlace simbólico al backup más reciente).*

2. Detener servicios que escriban en disco (Opcional pero recomendado)

Para evitar que los usuarios modifiquen archivos mientras se restaura: ```bash sudo systemctl stop nginx sudo killall -u dev1 dev2 dev3 dev4 2>/dev/null ```

3. Ejecutar la Restauración

Se utiliza `rsync` para volcar los datos desde la bóveda de seguridad al directorio de producción. El flag `--delete` asegura que los archivos corruptos actuales se borren y queden exactamente como en el backup. ```bash sudo rsync -av --delete /mnt/data_vault/backups/latest/ /home/greendevcorp/shared/ ```

4. Restaurar Permisos de Seguridad (SGID)

Las restauraciones pueden alterar los permisos especiales. Es crítico volver a forzar la política de la empresa: ```bash sudo chown -R root:greendevcorp /home/greendevcorp/shared sudo chmod -R 2770 /home/greendevcorp/shared sudo chmod +t /home/greendevcorp/shared ```

☑ Evidencia de Recuperación (Testing)

La recuperación ha sido verificada en un entorno de pruebas simulando un borrado accidental ("Fat-finger error").

Escenario de Prueba:

1. Se generó el archivo crítico `proyecto_final.sh` en `/home/greendevcorp/shared/`.
2. Se forzó la ejecución del timer: `sudo systemctl start admin-backup.service`.
3. Se comprobó la creación exitosa en: `/mnt/data_vault/backups/latest/proyecto_final.sh`.
4. El usuario simuló un borrado accidental: `rm -rf /home/greendevcorp/shared/*`.
5. Se ejecutó el comando de restauración del procedimiento superior.
6. **Resultado:** El archivo `proyecto_final.sh` fue restaurado con su contenido intacto y los permisos `greendevcorp` correctos en 1.2 segundos.

🧠 Ensayo de Reflexión del Equipo (Reflection Essay)

Equipo: Pau Planas y Jesus Martinez
Proyecto: Infraestructura y Servidor de Desarrollo - GreenDevCorp

---

1. What was the most challenging aspect of this project?

> *¿Cuál fue el aspecto más desafiante de este proyecto?*

Sin duda, nuestro mayor reto conjunto fue enfrentarnos al **"problema del huevo y la gallina"** en la automatización inicial y comprender la gestión de permisos a bajo nivel.

Durante el desarrollo, nos topamos con bloqueos críticos de los que era difícil salir: al intentar ejecutar la orquestación en una máquina virtual limpia, llegamos a perder los permisos de administrador (el temido error "*user NOT in sudoers*" y la pérdida de control sobre `su`).

Solucionar esto nos obligó a salir de nuestra zona de confort como equipo. Tuvimos que aprender a intervenir el proceso de arranque de Linux (GRUB), investigar cómo inyectar un entorno de rescate y lidiar con los identificadores criptográficos cambiantes de SSH (alertas de "*Man-in-the-middle*") al restaurar *snapshots* en VirtualBox. Fueron errores frustrantes, pero debatir y aplicar la solución juntos lo hizo tremendamente educativo.

2. What would you do differently if you started over?

> *¿Qué haríais diferente si volvierais a empezar?*

Si empezáramos el proyecto desde cero, adoptaríamos un enfoque de **programación mucho más defensiva**:

- **Idempotencia desde el día 1:** Al principio asumíamos que herramientas como `rsync` o ciertos comandos del sistema siempre estarían disponibles. Ahora entendemos que nuestros scripts deben comprobar proactivamente si una dependencia existe antes de intentar usarla o instalarla, evitando que el script se rompa a la mitad.
- **Automatización del despliegue base:** Implementaríamos el script `bootstrap.sh` mucho antes en la fase de diseño. Tener un único comando para descargar dependencias, clonar el repositorio de GitHub y lanzar la Semana 1 nos habría ahorrado horas de configuraciones repetitivas a mano en cada nueva prueba.

3. How has your understanding of system administration changed?

> *¿Cómo ha cambiado vuestra comprensión de la administración de sistemas?*

Antes de este proyecto, veíamos la administración de sistemas como la tarea reactiva de memorizar comandos aislados y "apagar incendios" manualmente en la terminal. Ahora hemos interiorizado el paradigma de la **Infraestructura como Código (IaC)** y el concepto de *Cattle vs. Pets* (Ganado vs. Mascotas).

Hemos comprendido que un servidor no es una máquina a la que hay que cuidar con mimo a mano; si nuestro entorno de producción se corrompe, no debemos pasar 5 horas arreglándolo. Simplemente lo destruimos, levantamos una máquina limpia y ejecutamos nuestro repositorio para levantar un clon exacto en minutos. Además, hemos descubierto que un sistema sin una buena observabilidad (Systemd Timers, límites Cgroups, monitorización de logs) es una caja negra muy peligrosa.

4. What’s one thing you’d want to learn more about?

> *¿Sobre qué aspecto os gustaría aprender más?*

Nos encantaría profundizar en la **orquestación y escalabilidad a gran escala**.

Aunque Bash y Shell Scripting nos han dado una base fundamental y potentísima para entender las entrañas del kernel de Linux, los permisos y los servicios, hemos notado sus limitaciones si tuviéramos que escalar esta misma infraestructura a 100 servidores simultáneos.

Nuestro siguiente paso natural sería aprender herramientas de configuración declarativa como **Ansible** o **Terraform** para aprovisionar flotas enteras de máquinas, o explorar la contenerización con **Docker** y

Kubernetes para independizar totalmente los entornos de desarrollo del sistema operativo base.

📘 Manual de Operaciones (Runbook) - GreenDevCorp

Este documento es la guía de referencia rápida para los ingenieros de operaciones de GreenDevCorp. Contiene los procedimientos estandarizados para las tareas del día a día, resolución de problemas comunes (Troubleshooting) y vías de escalado.

🔄 1. Tareas Comunes (How do I...?)

¿Cómo añado a un nuevo desarrollador al equipo?

Los desarrolladores deben pertenecer al grupo `greendevcorp` para heredar los permisos del directorio compartido y los límites de recursos (PAM). Para añadir un nuevo usuario (ej. `dev5`):

```bash

# 1. Crear el usuario y añadirlo al grupo principal

sudo useradd -m -g greendevcorp -s /bin/bash dev5

# 2. Asignar una contraseña temporal (se le pedirá cambiarla al entrar)

sudo passwd dev5 sudo chage -d 0 dev5 ```

### ¿Cómo gestiono la salida de un miembro del equipo?

Si un desarrollador abandona la empresa, **no debemos borrar su usuario inmediatamente** (`userdel`), ya que podríamos crear archivos huérfanos. Debemos bloquear su acceso:

```bash

# 1. Bloquear la contraseña de la cuenta

sudo usermod -L dev1

# 2. Expirar la cuenta inmediatamente para cerrar sesiones activas

sudo chage -E 0 dev1 ```

*(Nota: Gracias al SGID implementado en `/home/greendevcorp/shared`, el resto del equipo podrá seguir editando los archivos que creó este usuario).*

### ¿Cómo compruebo si los servicios funcionan correctamente?

Nuestra infraestructura se basa en `systemd`. Para auditar la salud del servidor:

- **Servidor Web (Nginx):** `sudo systemctl status nginx`
- **Sistema de Backups (Timers):** `sudo systemctl list-timers | grep backup`
- **Ver logs en tiempo real:** `sudo journalctl -u nginx -f`
- **Auditoría rápida (Script propio):** `check-services` (Comando global instalado en la Semana 2).

### ¿Cómo diagnostico un sistema lento?

No confíes solo en `top` o `htop`. Utiliza las herramientas de diagnóstico personalizadas que implementamos en la **Semana 3**:

1. **Identificar quién consume más recursos:** ```bash /opt/admin/week_3/09_top_consumers.sh ```
2. **Revisar si los límites de Cgroups están actuando:** ```bash systemd-cgtop ```
3. **Ver la jerarquía de procesos (¿Hay procesos zombie?):** ```bash /opt/admin/week_3/10_tree_relationships.sh ```

### ¿Cómo restauro desde un backup?

Tenemos un RTO (Recovery Time Objective) de minutos. Para restaurar el directorio compartido a su estado de la noche anterior:

```bash sudo rsync -av --delete /mnt/data_vault/backups/latest/ /home/greendevcorp/shared/ ```  *(Para el procedimiento completo de desastres y restauración de bases de datos, consulta el Plan de Disaster Recovery).*

---

## 🚨 2. Guía de Troubleshooting (Resolución de Problemas)

| Síntoma / Error | Causa Probable | Solución Inmediata |
|---|---|---|
| **"Permission denied" al editar un archivo en `/shared`** | El archivo fue copiado o creado sin heredar el grupo correcto. | Ejecutar `sudo chown -R root:greendevcorp /home/greendevcorp/shared` y asegurar que la carpeta tiene SGID (`chmod 2770`). |
| **Nginx aparece como "Failed" o reiniciándose en bucle** | Error de sintaxis en la configuración o el puerto 80 está ocupado. | Ejecutar `sudo nginx -t` para ver en qué línea está el error tipográfico. Corregir y hacer `sudo systemctl restart nginx`. |
| **"No space left on device" en `/mnt/data_vault`** | El disco virtual secundario de backups (5GB) se ha llenado. | Limpiar copias manuales antiguas o ajustar la política de retención del script de *rsync* en la Semana 5. |
| **Usuario administrador no puede ejecutar `sudo`** | La sesión no se ha refrescado o el usuario perdió el grupo. | Entrar como root puro (`su -`) y ejecutar `usermod -aG sudo <usuario>`. Salir y volver a iniciar sesión. |

---

## 📞 3. Procedimientos de Escalado (Escalation)
```

Si un problema crítico ocurre en producción y no puede resolverse en **30 minutos** usando este Runbook, sigue este orden estricto de escalado:

1. **Nivel 1 (Operaciones Básicas):** Reiniciar el servicio afectado (`systemctl restart <servicio>`) y verificar espacio en disco (`df -h`).
2. **Nivel 2 (Diagnóstico Avanzado):** Buscar errores críticos a nivel de kernel (`dmesg -T | tail -n 20`) y revisar los logs generales del sistema (`journalctl -xe`).
3. **Nivel 3 (Escalado Crítico):** Si el servidor está inaccesible por SSH, Nginx no levanta, o hay pérdida de datos confirmada:
    - **Contactar inmediatamente** a los Administradores de Sistemas Senior (Pau Planas / [Nombre Compañero]).
    - Abrir un ticket de *Incidencia Crítica (P1)* en el repositorio de GitHub detallando los últimos comandos ejecutados.